

Deployment Guide

DocsForge builds static HTML that can be deployed to any static hosting platform. This guide covers the most popular options.

Quick Comparison

Platform	Cost	Custom Domain	HTTPS	CI/CD	Best For
GitHub Pages	Free	✓	✓	GitHub Actions	Open source projects
Netlify	Free tier	✓	✓	Git push	Prototyping, JAMstack
Vercel	Free tier	✓	✓	Git push	Next.js, fast deploys
Cloudflare Pages	Free	✓	✓	Git push	Speed, CDN
GitLab Pages	Free	✓	✓	GitLab CI	GitLab users
AWS S3 + CloudFront	Pay per use	✓	✓	GitHub Actions	Enterprise, scale
Firebase Hosting	Free tier	✓	✓	GitHub Actions	Google ecosystem
Surge.sh	Free	✓	✓	CLI	Quick deploys
Render	Free tier	✓	✓	Git push	Static sites, fast CDN

Platform	Cost	Custom Domain	HTTPS	CI/CD	Best For
DigitalOcean App Platform	Pay per use	✓	✓	GitHub Actions	Full control, scalable
Docker + Nginx	Server cost	✓	✓	Any	Self-hosted
Caddy	Server cost	✓	Auto	Any	Self-hosted, easy TLS

GitHub Pages

The simplest option for open-source projects. Free, reliable, integrated with GitHub.

GitHub Actions (Recommended)

Create `.github/workflows/docsforge-pages.yml`:

```
name: Deploy Docs

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install docsforge
      - run: docsforge build
      - uses: actions/upload-pages-artifact@v3
```

```

    with:
      path: site

  deploy:
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    runs-on: ubuntu-latest
    needs: build
    steps:
      - uses: actions/deploy-pages@v4
        id: deployment

```

Alternative: peaceiris/actions-gh-pages

A simpler single-job workflow that pushes to the `gh-pages` branch:

```

name: Deploy Docs

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install docsforge
      - run: docsforge build
      - uses: peaceiris/actions-gh-pages@v4
        with:
          github_token: ${ secrets.GITHUB_TOKEN }
          publish_dir: ./site

```

Settings

1. Go to **Settings** → **Pages**
2. Source: **GitHub Actions**
3. Push the workflow file — your site deploys automatically

Custom Domain

1. Add `CNAME` file to `docs/` (or `site/` after build):

```
docs.example.com
```

2. In repository settings: **Pages** → **Custom domain**

3. Add DNS records:

- `docs.example.com` → CNAME → `yourusername.github.io`
-

Netlify

Great for prototyping and JAMstack sites. Drag-and-drop or git-based deploys.

Method 1: Git-based (Recommended)

1. Push your repo to GitHub/GitLab/Bitbucket
2. Log in to [Netlify](#)
3. **Add new site** → **Import from Git**
4. Select your repository
5. Build settings:
 - Build command: `pip install docsforge && docsforge build`
 - Publish directory: `site`
6. Deploy!

Method 2: CLI

```
# Install Netlify CLI
npm install -g netlify-cli

# Build your docs
docsforge build

# Deploy
netlify deploy --dir=site --prod
```

netlify.toml

```
[build]
  command = "pip install docsforge && docsforge build"
  publish = "site"

[[redirects]]
  from = "/*"
  to = "/404.html"
  status = 404
```

Vercel

Fast deploys, great for frontend projects. Similar to Netlify.

Git-based

1. Push to GitHub
2. Log in to [Vercel](#)
3. **Add New Project** → **Import**
4. Framework preset: **Other**
5. Build command: `pip install docsforge && docsforge build`
6. Output directory: `site`

`vercel.json`

```
{
  "builds": [
    {
      "src": "docsforge.yml",
      "use": "@vercel/static-build",
      "config": {
        "distDir": "site"
      }
    }
  ]
}
```

Cloudflare Pages

Fastest CDN, generous free tier. Great for global performance.

Git-based

1. Push to GitHub/GitLab
2. Log in to [Cloudflare Dashboard](#)
3. **Pages** → **Create a project**
4. Connect your git repository
5. Build settings:
 - Build command: `pip install docsforge && docsforge build`
 - Build output directory: `site`

6. Deploy

Cloudflare Pages Functions

For custom headers or redirects, add `_headers` or `_redirects` to `docs/`:

```
# docs/_headers
/*.*html
  X-Frame-Options: DENY
  X-Content-Type-Options: nosniff
```

GitLab Pages

Integrated with GitLab CI. Great for GitLab users.

`.gitlab-ci.yml`

```
image: python:3.11

pages:
  script:
    - pip install docsforge
    - docsforge build
  artifacts:
    paths:
      - site
  only:
    - main
```

Your site will be at `https://username.gitlab.io/projectname`.

AWS S3 + CloudFront

Enterprise-grade hosting. Scales to any traffic level.

S3 Bucket Setup

1. Create S3 bucket (e.g., `docs.example.com`)
2. Enable **Static website hosting**
3. Upload your `site/` directory:

```
aws s3 sync site/ s3://docs.example.com --delete
```

4. Set bucket policy to allow public read:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::docs.example.com/*"
    }
  ]
}
```

CloudFront CDN

1. Create CloudFront distribution
2. Origin: Your S3 bucket
3. Viewer protocol policy: **Redirect HTTP to HTTPS**
4. Add custom domain in **Alternate domain names**
5. Use AWS Certificate Manager for SSL

GitHub Actions + S3

```
- name: Deploy to S3
  run: aws s3 sync site/ s3://docs.example.com --delete
  env:
    AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
```

Firebase Hosting

Part of Google Firebase. Good if you're already in the Google ecosystem.

Setup

```
# Install Firebase CLI
npm install -g firebase-tools

# Login
firebase login

# Init project
firebase init hosting
```

```
# Select "Configure as a single-page app?" → No
# Set public directory → site

# Build and deploy
docsforge build
firebase deploy
```

firebase.json

```
{
  "hosting": {
    "public": "site",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [
      {
        "source": "**",
        "destination": "/404.html"
      }
    ]
  }
}
```

Surge.sh

Simplest possible deployment. Perfect for quick previews.

```
# Install Surge
npm install -g surge

# Build and deploy
docsforge build
surge site/ docs-example.surge.sh

# Custom domain
surge site/ docs.example.com
```

Render

[Render](#) offers a generous free tier for static sites with global CDN, automatic HTTPS, and git-based deploys.

Git-based (Recommended)

1. Push your repo to GitHub/GitLab
2. Log in to [Render Dashboard](#)

3. New + → Static Site

4. Connect your repository

5. Build settings:

- Build command: `pip install docsforge && docsforge build`
- Publish directory: `site`

6. Create Static Site

Render auto-deploys on every push to the main branch.

Custom Domain

1. Go to your static site's **Settings** → **Custom Domain**
 2. Add your domain
 3. Update DNS (Render provides the target CNAME)
-

DigitalOcean App Platform

[DigitalOcean App Platform](#) provides managed static hosting with automatic HTTPS, CDN, and git-based deployments.

Git-based (Recommended)

1. Push your repo to GitHub
2. Log in to [DigitalOcean](#)
3. **Apps** → **Create App**
4. Connect your repository
5. Select the branch (`main`)
6. Edit the plan (Static Site starts at \$0 — free tier available)
7. Build settings:
 - Build command: `pip install docsforge && docsforge build`
 - Output directory: `site`
8. **Review and Create**

GitHub Actions + DigitalOcean

For more control, build locally and deploy via the DigitalOcean CLI:

```
- name: Deploy to DigitalOcean App Platform
  uses: digitalocean/app_action@v1
  with:
    app_name: my-docs
    token: ${ secrets.DIGITALOCEAN_ACCESS_TOKEN }
```

Docker + Nginx

Self-hosted option. Full control over the server.

Dockerfile

```
FROM python:3.11-slim AS builder

WORKDIR /app
COPY . /app
RUN pip install docsforge && docsforge build

FROM nginx:alpine
COPY --from=builder /app/site /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
```

nginx.conf

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    error_page 404 /404.html;
}
```

Build and Run

```
docker build -t my-docs .
docker run -p 8080:80 my-docs
```

Caddy

Modern, easy-to-configure web server with automatic HTTPS.

Caddyfile

```
docs.example.com {  
    root * /var/www/docs  
    file_server  
    try_files {path} {path}/ /404.html  
}
```

Deploy

```
# Build docs  
docsforge build  
  
# Copy to server  
rsync -avz site/ user@server:/var/www/docs/  
  
# Start Caddy  
caddy run --config Caddyfile
```

Caddy automatically provisions Let's Encrypt certificates. No manual SSL setup.

Platform-Specific Tips

GitHub Pages

- Use `site_url: https://username.github.io/repo-name` in config
- For user/org sites (`username.github.io`), set `site_url` accordingly
- Enable "Enforce HTTPS" in repository settings

Netlify

- Add `_redirects` file for SPA routing:

```
/* /404.html 404
```

- Use branch deploys for previewing PRs

Vercel

- Set `site_url` to your Vercel domain

- Use `vercel --prod` for production deploys

Cloudflare Pages

- Enable "Always Use HTTPS"
- Use Cloudflare's caching for static assets

AWS S3

- Enable versioning if you want rollback capability
- Use `aws s3 sync --delete` to remove old files
- Consider S3 Transfer Acceleration for global audiences

Render

- Set `site_url` to your Render domain (e.g. `https://my-docs.onrender.com`)
- Auto-deploys on every push to the connected branch
- Free tier includes 100 GB bandwidth/month

DigitalOcean

- Static Sites start free with 1 GB storage
 - Set `site_url` to your App Platform domain
 - Use the DigitalOcean API token in GitHub Actions for automated deploys
-

Continuous Deployment

Trigger Deploys on Every Push

All platforms support automatic deploys when you push to git. The general pattern:

1. **Build:** Install DocsForge, build the site
2. **Deploy:** Upload `site/` to your host
3. **Invalidate:** Clear CDN cache (if using CDN)

Example: GitHub Actions + Any Platform

```
name: Deploy
on:
  push:
```

```
branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install docsforge
      - run: docsforge build
      # Add your deploy step here
```

Troubleshooting

Issue	Platform	Solution
404 on refresh	SPA hosts	Configure fallback to <code>index.html</code>
CSS not loading	All	Check <code>site_url</code> matches actual domain
Search not working	All	Ensure <code>search</code> plugin is enabled
Build fails	CI/CD	Pin Python version, use <code>pip install docsforge</code>
Slow deploys	All	Use <code>.gitignore</code> to exclude large files
Cache issues	CDN	Add cache-busting query strings or purge CDN

Next Steps

- [Usage Guide](#) — Day-to-day DocsForge usage
- [Migration Guide](#) — Moving from MkDocs/Material
- [Changelog](#) — What's new